For the Alert Generation System, I chose to use association arrows between the classes because they only reference each other. For example AlertGenerator uses ThresholdConfig to check limits, but ThresholdConfig is independent because it can exist on its own, so therefore, an association made more sense than composition. I labeled this relationship "uses" and gave it a 1 to 1 multiplicity because each AlertGenerator works with exactly one ThresholdConfig at a time.The relationship between AlertGenerator and Alert is labeled "creates" with a 1 to 0..* multiplicity. I chose 0..* on the Alert side because an AlertGenerator might not create any alerts at all if the patient is healthy, but it could also create many alerts over time. I connected AlertGenerator to AlertManager with a 1 to 1 association arrow, which was labeled "sends to" because there is one manager handling all dispatching in the system. Then AlertManager connects to MedicalStaff with a 1 to 0..* multiplicity because one manager can notify multiple staff members depending on who is on duty. I kept all relationships as plain associations rather than composition because none of these classes fully own each other. They work together and pass data between each other but they can also all exist independently. This keeps the design loosely coupled which makes it more flexible and therefore easier to change individual parts of the system without breaking everything else.

For the Data Storage System, I chose to use both association arrows and composition arrows between the classes because some of them only reference each other, but others have strong ownership. For example, the class Data Storage and Patient Data are connected with a composition arrow and have 1 to 0..* multiplicity because Patient Data is fully dependent on the Data Storage class. That is because PatientData records are owned by DataStorage and cannot exist without it. If the storage system were to be deleted or destroyed, then the records in PatientData would also be lost. I connected DataRetriever to DataStorage with an association arrow which was labeled “queries” and a 1 to 1 multiplicity. I chose an association arrow because both can exist without the other and have separate concerns. I also added AccessControl as a separate class connected to DataRetriever with a “checks” label and an association arrow, as well as a 1 to 1 multiplicity. The reason I chose to use an association arrow instead of a composition arrow is because DataRetriever does not depend on or own DataStorage, instead it just uses it to look things up. Both classes have separate concerns and can exist without each other.

For the Patient Identification System, I chose to use both association arrows and composition arrows between the classes. That is because some of them only reference each other, but others have strong ownership over others. For example, I chose to use a composition arrow to represent the relationship between PatientDatabase and HospitalPatient with a 1 to 0..* multiplicity. That is because HospitalPatient records are fully owned by PatientDatabase, and therefore cannot exist without it. The PatientIdentifier class was connected to the PatientDatabase via an association arrow, which was labeled “looks up in”, as well as a 1 to 1 multiplicity. The two classes are not dependent on each other, instead they have separate concerns and can exist without one another. PatientIdentifier does not own the database, it just queries it to find a match for incoming data. I also connected PatientIdentifier to IdentityManager with a "reports to" association arrow and a 1 to 1 multiplicity. The reason for this design choice is that when PatientIdentifier finds some sort of mismatch, it needs somewhere to send that problem. IdentityManager is responsible for handling those edge cases, so separating it into its own class keeps the error handling

logic away from the matching logic. I connected IdentityManager to HospitalPatient with a "verifies" association arrow and a 1 to 0..* multiplicity because IdentityManager is not owned by HospitalPatient, instead both classes can exist without one another.

For the Data Access Layer, I chose to use both association and inheritance arrows. The reason for this is that DataListener is an abstract class because all three listener types share the same basic behaviour. Therefore by putting these shared methods in one abstract class and having TCPDataListener, WebSocketDataListener, and FileDataListener all inherit from it, I avoided repeating the same code multiple times. This is why I used inheritance arrows pointing from each subclass up to DataListener. Each of the subclasses also has its own unique methods on top of what they inherit. I connected all three subclasses to DataParser with a label "sends raw data" and association arrows, because regardless of where the data comes from it all needs to be parsed into a usable way before the system can do anything with it. I connected DataParser to DataSourceAdapter with a "passes parsed data" label and an association arrow because once the data is parsed it needs to be sent to the rest of the system.